



Widgets & Layout.

Yesterday you built a 5-line app. Today we add the 5 layout primitives that turn a stack of widgets into a real dashboard — sidebars, columns, tabs, expanders, and the session state that makes it all remember.

PROGRAM

Python Internship

LED BY

Zlaark

SESSION

5 of 24

How we'll spend our 90 minutes today.

Five layout primitives, one session-state preview, and a personal-info-form exercise. The day your apps stop being a vertical stack and start looking intentional.

DURATION
90 min

00:00 – 00:10



Day 4 Recap & Q&A

Streamlit run-loop, the 5-step app, the unit converter. Quick check.

00:10 – 00:20



Widget Keys

Why every widget should have a unique key. The habit that pays off in Phase 4.

00:20 – 00:45



The 5 Layout Primitives

`st.columns`, `st.sidebar`, `st.tabs`, `st.expander`,
`st.container` / `st.empty`. One slide each, typed live.

00:45 – 01:00



session_state Preview

The #1 question from Day 4: how do I make a counter that actually persists across reruns?

01:00 – 01:20



Exercise: Personal Info Form

Build a form with `st.form`: name, email, age slider, gender radio, submit button. The form pattern groups will use in Phase 4.

01:20 – 01:30



Q&A and Day 6 Preview

Open floor, then a quick look at Working With Data (pandas).

Yesterday you built your first app. Today we make it look intentional.

Quick check that Day 4's app runs. If it doesn't, we fix it now — today builds directly on the run-loop mental model.

WHAT WE COVERED YESTERDAY DAY 04

- ✓ **The Streamlit run-loop** — every click re-runs your whole script top to bottom
- ✓ **Built a 5-step app:** `st.title` → `text_input` → `button` → `balloons` → `else`
- ✓ **The mental model:** widgets are functions that return values, not events
- ✓ **The `st.*` cheatsheet** — 10 commands that cover 90% of use cases
- ✓ **Built a unit converter** with `selectbox` + `number_input` + live result

CHECK YOUR LAPTOP NOW VERIFY

```
# You should have Day 4's app files:
$ ls day4/
01_app_step1_title.py
02_app_step2_text_input.py
...
05_app_step5_else.py
07_converter_solution.py

# Run the final app — should open in browser:
$ streamlit run 05_app_step5_else.py
# → Browser opens at localhost:8501
# → Type your name, click button, balloons fly
```

⚠ **IF THE APP DOESN'T RUN** — raise your hand. Common issue: forgot to activate `.venv` first. Run `source .venv/bin/activate` (macOS) or `.venv\Scripts\Activate.ps1` (Windows), then retry.

• CONCEPT • WIDGET KEYS

Every widget should have a **key**. It costs nothing today. It saves you in Phase 4.

A key is a unique string name for a widget. Without it, Streamlit generates one automatically. With it, you control the widget programmatically — read its value, update it, reset it.

WITHOUT A KEY

THE DEFAULT

```
# Streamlit auto-generates a key
name = st.text_input("Your name")
# → internally key = "text_input_0"

age = st.slider("Age", 0, 100, 20)
# → internally key = "slider_0"

# Problems:
# - Key changes if you reorder widgets
# - Can't access the widget from session_state
# - Can't programmatically update or reset it
# - Hard to debug when two widgets clash
```

Works for a 10-line demo. Falls apart in a 200-line project.

WITH A KEY

THE HABIT

```
# You name the widget explicitly
name = st.text_input("Your name", key="user_name")
age = st.slider("Age", 0, 100, 20, key="user_age")

# Now you can:
# - Read it from anywhere: st.session_state.user_name
# - Update it: st.session_state.user_age = 25
# - Reset it: del st.session_state.user_name
# - Reorder widgets freely (key stays stable)
# - Debug by name, not by index
```

Same code, plus one keyword arg. Pays off the moment your app grows past 50 lines.

RULE OF THUMB If a widget will EVER be read, updated, or reset by code (not just the user), give it a **key**. The cost is **one keyword argument**. The benefit is **full programmatic control**.

Five primitives. Every Streamlit layout is a combination of these.

You already know widgets — title, text_input, button, slider. Layout primitives organize WHERE those widgets appear on the page. Learn these five, and you can build any dashboard.

01

`st.columns(n)`



Side-by-side panels



USE WHEN

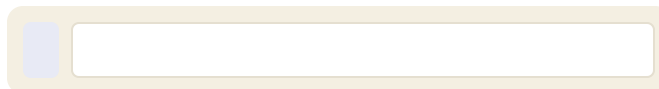
You want inputs and outputs on the same screen, not stacked vertically.

02

`st.sidebar`



Left sidebar for controls



USE WHEN

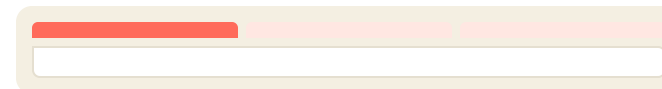
Controls go in the sidebar, output stays in the main area. The dashboard pattern.

03

`st.tabs(names)`



Tabbed sections



USE WHEN

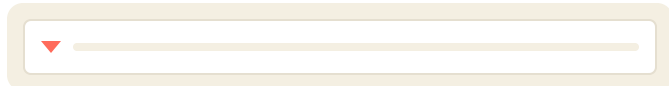
You have multiple views (e.g. “Input” / “Chart” / “Raw Data”) and want only one visible at a time.

04

`st.expander(label)`



Collapsible section



USE WHEN

Advanced options or long reference text that should be hidden by default, expanded on click.

05

`st.container` / `st.empty`



Dynamic content placeholder



USE WHEN

You need to insert content at a specific position, or update content in place (streaming, progress bars).

💡 THE PATTERN

Pick ONE primary layout primitive per page. Combine others inside it.

Example: a `sidebar` with a `form`, whose submit renders a `tabs` view with `columns` inside each tab.

st.columns — split the page side-by-side.

The most-used layout primitive. Takes a number, returns a list of column objects. Use them as context managers (`with col1:`) to place widgets inside.

```

import streamlit as st

st.title("Two Columns Demo")

# Split the page into 2 columns
col1, col2 = st.columns(2)

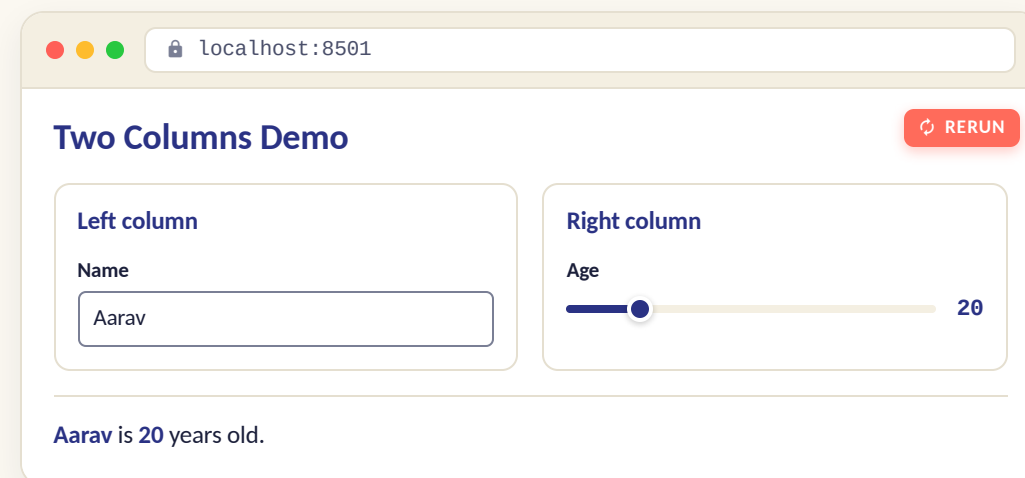
# Put widgets INSIDE each column
with col1:
    st.write("**Left column**")
    name = st.text_input("Name", key="name")

with col2:
    st.write("**Right column**")
    age = st.slider("Age", 0, 100, 20, key="age")

# Below the columns — full width
st.divider()
st.write(f"{name} is {age} years old.")

# Pro tip: uneven columns with ratios
# col1, col2 = st.columns([1, 3]) # col2 is 3x wider
    
```

BROWSER PREVIEW • WHAT THE STUDENT SEES



Inputs and outputs live on the same screen — no scrolling required.

1 **st.columns(2)** — Returns a tuple of 2 column objects. Unpack into variables. Use `st.columns([1, 3])` for uneven ratios.

2 **with col1:** — Context manager. Anything indented under it renders inside that column. Standard Python `with` syntax.

USE WHEN You want inputs and outputs visible on the same screen, not stacked vertically. The most common dashboard pattern.

st.sidebar — controls on the left, output on the right.

The dashboard pattern. Controls go in the sidebar (sliders, dropdowns, inputs). The main area shows the result. Used by every analytics dashboard at every real company.

```

app.py PYTHON

import streamlit as st

st.title("Dashboard Demo")

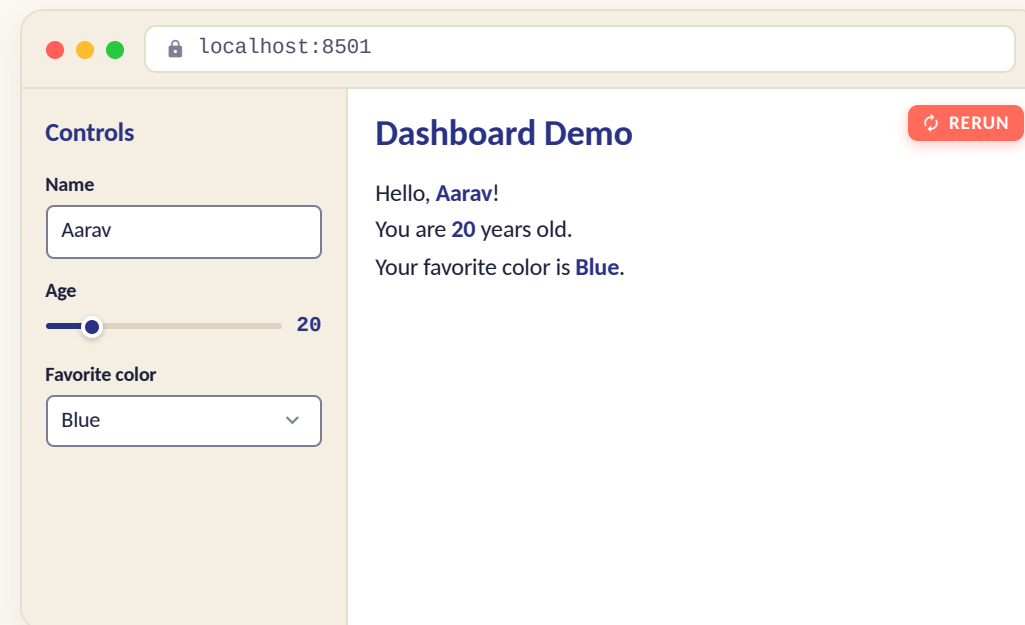
# Controls in the SIDEBAR
with st.sidebar:
    st.write("**Controls**")
    name = st.text_input("Name", key="name")
    age = st.slider("Age", 0, 100, 20, key="age")
    color = st.selectbox("Favorite color",
                        ["Red", "Green", "Blue"], key="color")

# Output in the MAIN area
st.write(f"Hello, **{name}**!")
st.write(f"You are **{age}** years old.")
st.write(f"Your favorite color is **{color}**.")

# Alternative shorthand (no `with` block):
# name = st.sidebar.text_input("Name")
# — works for a single widget

```

BROWSER PREVIEW • SIDEBAR + MAIN AREA



Change a sidebar control — only the main area re-renders. The sidebar stays put.

- 1 **with st.sidebar:** — Same context-manager pattern as `st.columns`. Everything indented renders in the left sidebar.
- 2 **Shorthand:** `st.sidebar.text_input(...)` — For a single widget, you can skip the `with` block. Picks are stylistic — `with` is cleaner for multiple widgets.
- 3 **Main area = default** — Anything NOT inside a sidebar/columns/tabs context renders in the main area, full width.

USE WHEN You have controls that don't change often + output that updates frequently. The sidebar stays put, the main area re-renders. This is how every real dashboard works.

st.tabs — multiple views. st.expander — hide the optional.

Both primitives organize content that doesn't all need to be visible at once. Tabs switch between views. Expanders show/hide a section.

03 | st.tabs(names)

Split content into **tabbed sections**. Only one tab visible at a time.

```
tab1, tab2, tab3 = st.tabs(
    ["Input", "Chart", "Raw Data"]
)

with tab1:
    name = st.text_input("Name")
with tab2:
    st.write("Chart goes here")
with tab3:
    st.write("Raw data goes here")
```

USE WHEN You have multiple views of the same data (input form, visualization, raw table). Users pick which to see.

SKETCH • RENDERED



04 | st.expander(label)

Collapsible section. Hidden by default, expands on click.

```
with st.expander("Advanced options"):
    st.write("These are rare.")
    threshold = st.slider(
        "Threshold", 0.0, 1.0, 0.5)
    use_cache = st.checkbox(
        "Use cache", value=True)

# Collapsed by default.
# Set expanded=True to open by default:
# st.expander("Help", expanded=True)
```

USE WHEN Advanced/rare controls, long help text, reference material. Keep the main page clean.

SKETCH • COLLAPSED (DEFAULT)

▶ Advanced options

click to expand → reveals sliders & checkboxes

PATTERN Tabs are for switching **VIEWS**. Expanders are for hiding **DETAILS**. Both reduce visual clutter, but for different reasons.

st.container + st.empty — insert content where you want, update it later.

The advanced primitives. Most apps won't need these on Day 5, but you'll see them in every chatbot, progress bar, and streaming-output app from Day 12 onward.

05a | st.container()

A **placeholder** you can insert widgets into, out of order.

```
# Insert a container at a specific position
holder = st.container()

# ... other widgets render here ...
st.write("This renders first.")

# Now insert INTO the container (above)
with holder:
    st.write("This renders ABOVE 'first'")
    st.write("Even though it's defined later.")
```

USE WHEN You need to render content at a position determined before you know what the content is. Rare in beginner apps, common in chat UIs.

🔖 You'll meet this again on [Day 13 • chat UIs](#)

05b | st.empty()

A **single-slot placeholder** you can update in place.

```
# Reserve a spot on the page
status = st.empty()

# Later, update it (replaces content)
status.write("Loading...")

# Even later, update again
import time
time.sleep(1)
status.write("Done!")

# Common pattern: progress bars
# bar = st.progress(0)
# for i in range(100):
#     bar.progress(i + 1)
```

USE WHEN Streaming output, progress bars, loading spinners, chat messages that appear over time.

🔖 You'll meet this again on [Day 12 • AI streaming](#) • [Day 22 • loading states](#)

TODAY VS LATER You probably won't use these in today's exercise. But you'll see them again on [Day 12](#) (AI text streaming) and [Day 22](#) (deployment loading states). Bookmark this slide.

How do I make a counter that actually persists across reruns?

Day 4's gotcha: variables reset every run because the script re-runs top to bottom. The answer: `st.session_state`. A dictionary that survives across reruns.

THE BUG • WITHOUT SESSION_STATE

Broken

```
# counter.py – BROKEN
import streamlit as st

counter = 0 # resets every run!

if st.button("Click me"):
    counter += 1

st.write(f"Count: {counter}")

# Click the button:
# → Script re-runs
# → counter = 0 (reset!)
# → counter += 1 → counter = 1
# → st.write shows "Count: 1"

# Click again:
# → Script re-runs
# → counter = 0 (reset!)
# → counter += 1 → counter = 1
# → st.write STILL shows "Count: 1"
```

This is the run-loop in action. Every rerun is a fresh start.

THE FIX • WITH SESSION_STATE

Persisted

```
# counter.py – FIXED
import streamlit as st

# Initialize once, then persist
if "counter" not in st.session_state:
    st.session_state.counter = 0

if st.button("Click me"):
    st.session_state.counter += 1

st.write(f"Count: {st.session_state.counter}")

# Click the button:
# → Script re-runs
# → session_state.counter is STILL 1 (persisted!)
# → += 1 → counter = 2
# → st.write shows "Count: 2"

# Click again:
# → counter = 3, then 4, then 5...
# It works.
```

`session_state` is a dict that survives reruns. Initialize once, then read/write freely.

THE PATTERN — `if key not in st.session_state: st.session_state[key] = default`. Then read/write `st.session_state[key]` anywhere in your app. This is the only way to persist state across the run-loop.

Bookmark this. Five primitives, five use cases.

The skill is not knowing the API — it's knowing which primitive fits the problem. Here's the decision tree.



st.columns

USE WHEN Inputs and outputs should be visible together on one screen (e.g. form on left, preview on right).

DON'T USE WHEN You have more than 3–4 widgets per column — it gets cramped. Use sidebar instead.



st.tabs

USE WHEN Multiple views of the same data (Input / Chart / Raw Data). Only one needs to be visible at a time.

DON'T USE WHEN Users need to see two views simultaneously — tabs hide one. Use columns instead.



st.container / st.empty

USE WHEN Streaming output, progress bars, chat messages, loading states. Content that changes over time.

DON'T USE WHEN Static content — st.write is simpler. These primitives add complexity, use only when needed.



st.sidebar

USE WHEN Controls don't change often, output updates frequently. The dashboard pattern.

DON'T USE WHEN You have only 1–2 controls — a sidebar feels empty. Just put them at the top of the main area.



st.expander

USE WHEN Advanced options, help text, reference material. Hidden by default, expanded on demand.

DON'T USE WHEN The content is essential to the workflow — hiding it confuses users. Keep it visible.

♦ DECISION TREE

Need controls + output together? → **columns** .

Controls don't change? → **sidebar** .

Multiple views? → **tabs** .

Hide optional? → **expander** .

Content updates over time? → **container/empty** .

03 / 03

Build a form.

The Phase-4 pattern.

Today's exercise uses **st.form** — the batch-input pattern every group will use in their project. Name, email, age, gender, submit. The shape of a real data-collection app.

1 BRIEF · 5 MIN

2 BUILD · 15 MIN

3 EXTEND · 5 MIN

Build a personal info form with `st.form`.

THE BRIEF

Task

Build a Streamlit app called `form.py` that collects personal info using `st.form`. Fields: name (text input), email (text input), age (slider 0–100), gender (radio: Male/Female/Other/Prefer not to say), and a Submit button. On submit, show a summary card with all the entered data.

REQUIREMENTS

- ✓ Use `with st.form("my_form"):` to wrap the inputs
- ✓ Every widget has a unique `key` (the habit from slide 5)
- ✓ The form has a submit button: `submitted = st.form_submit_button("Submit")`
- ✓ On submit (`if submitted:`), show the summary using `st.write` or `st.json`
- ✓ Add a clear button that resets the form (use `st.session_state` + `del` — preview of slide 11)

BY THE END

Every student has a working form that collects 4 fields and shows a summary on submit. The form pattern groups will reuse in Phase 4.

A form batches multiple inputs into a single submit. Unlike regular widgets (which re-run on every change), a form only re-runs when the user clicks Submit. The pattern for data collection.

STARTING SCAFFOLD

form.py

```
# form.py – starting scaffold
import streamlit as st

st.title("Personal Info Form")

# 1. Define the form
with st.form("personal_info"):
    st.write("**Enter your details**")
    name = st.text_input("Name", key="name")
    email = st.text_input("Email", key="email")
    age = st.slider("Age", 0, 100, 20, key="age")
    gender = st.radio(
        "Gender",
        ["Male", "Female", "Other", "Prefer not to say"],
        key="gender"
    )

# Every form needs a submit button
submitted = st.form_submit_button("Submit")

# 2. Handle the submit
if submitted:
    st.success("Form submitted!")
    st.write(f"**Name:** {name}")
    st.write(f"**Email:** {email}")
    st.write(f"**Age:** {age}")
    st.write(f"**Gender:** {gender}")
```

RUN IT — `streamlit run form.py`. Fill the form, click Submit. Notice: typing in the fields does NOT trigger a rerun. Only Submit does. That's the form pattern.

Finished early? Add validation. Or read the pitfalls.

Three extension ideas for fast students. Three pitfalls everyone hits with forms and `session_state`.

EXTENSION IDEAS

For Fast Students

1 Add a Clear button

Below the form, add `if st.button("Clear"): del st.session_state["name"]; del st.session_state["email"]` etc. This resets all fields. Uses the `session_state` pattern from slide 11.

2 Add validation

Inside the `if submitted:` block, check that name is not empty and email contains `"@"`. If invalid, show `st.error("Please enter a valid email")` and don't show the summary. The pattern for any data-collection app.

3 Save to a list with `session_state`

Maintain a list `st.session_state.submissions = []` and append each submitted form as a dict. Show all past submissions below the form using `st.dataframe(st.session_state.submissions)`. A mini database — without a database.

PITFALLS YOU'LL HIT

Common Traps



Form widgets don't re-run on every change

Unlike regular widgets, typing in a form field does NOT trigger a rerun. Only `st.form_submit_button` does. This is the point of forms — batch the inputs. But it confuses students expecting live updates.



`session_state` key must exist before you read it

You can't do `st.session_state.counter += 1` without initializing it first. Always use `if "counter" not in st.session_state: st.session_state.counter = 0` at the top. Otherwise: `KeyError`.



Widget key collisions

Two widgets with the same `key="name"` will conflict. Streamlit raises a `DuplicateWidgetID` error. Keys must be unique across the whole app. Use descriptive names: `user_name`, `contact_email`, not `name`, `email`.

IF YOU'RE STUCK — Pair with a neighbour. Read the error message out loud. Raise your hand. We move at the room's pace.

Today your apps stopped being a vertical stack.

Tomorrow we add data — pandas, dataframes, filtering, sorting. Your apps start remembering things.

OPEN THE FLOOR

Ask Now

- ✓ "Which layout primitive felt most natural — columns, sidebar, tabs?"
- ✓ "Did session_state click, or does it still feel like magic?"
- ✓ "Anything from the form exercise that didn't work?"

BETWEEN SESSIONS

Get Unstuck

- ✓ **Email** • hello@zlaark.com
For layout questions, paste your app.py and a screenshot.
- ✓ **Web** • www.zlaark.com
Program updates and resources.
- ✓ **In-session** • **raise a hand**
No question is too small. We move at the room's pace.

WHAT'S NEXT • DAYS 6-8

Phase 2 Continues

- ✓ **Day 6** — Working With Data. pandas, dataframes, CSV loading, filtering.
- ✓ **Day 7** — Charts & Dashboards. Line charts, bar charts, a real dashboard.
- ✓ **Day 8** — Solo challenge. The gate before group formation on Day 15.